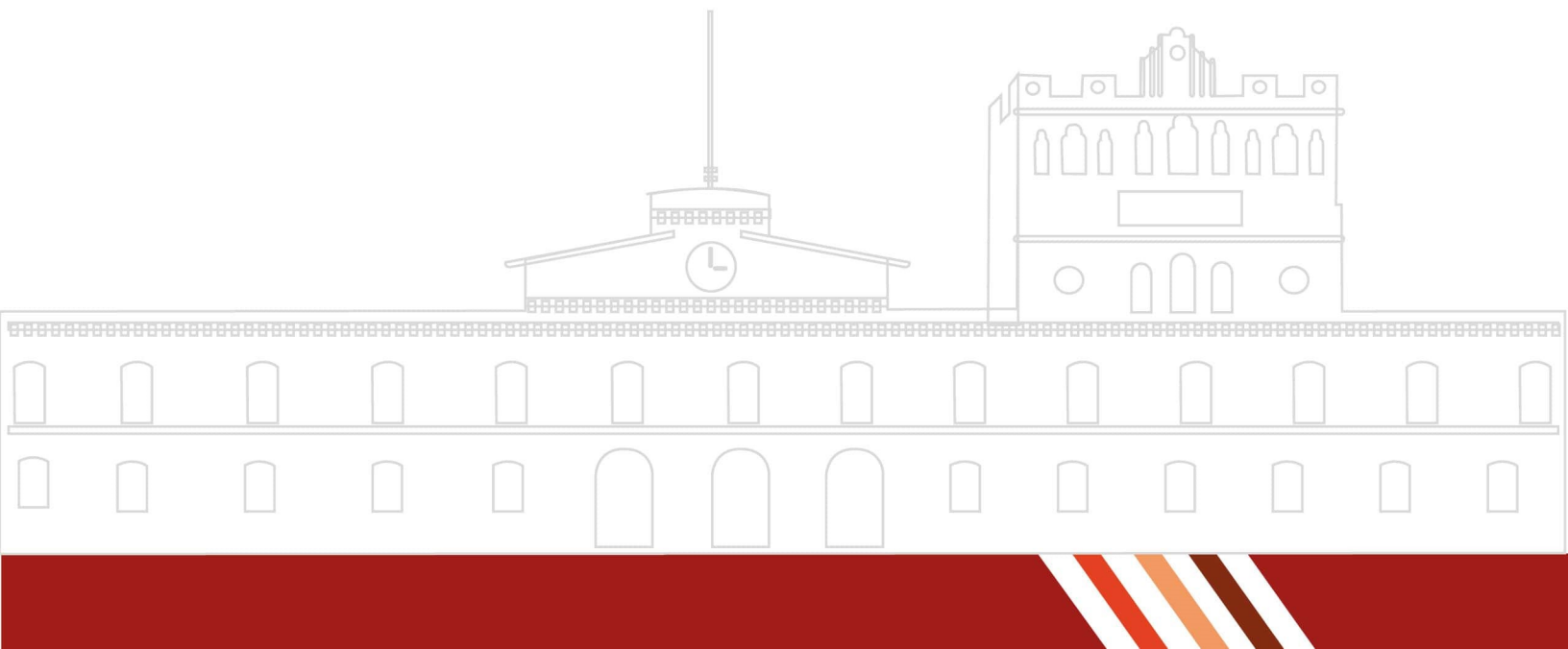


REPORTE DE PRÁCTICA NO. 3

Construcción de un Autómata Finito Determinista

ALUMNO: PINEDA FLORES JAVIER ALEJANDRO

Dr. Eduardo Cornejo-Velázquez



Introducción

Los Autómatas Finitos Deterministas (AFD) son modelos matemáticos utilizados para reconocer cadenas de símbolos pertenecientes a un lenguaje determinado. Estos autómatas funcionan mediante un conjunto finito de estados y una función de transición que determina el estado siguiente dependiendo del símbolo de entrada.

En esta práctica se desarrolla un programa en un lenguaje de programación de alto nivel capaz de implementar un AFD y analizar diferentes cadenas de símbolos para determinar si pertenecen o no al lenguaje reconocido por el autómata.

La construcción del programa permite relacionar los conceptos teóricos de los autómatas con su implementación computacional, facilitando la comprensión de los estados, el alfabeto, las transiciones y los estados de aceptación.

Marco teórico

Un Autómata Finito Determinista (AFD) es una máquina abstracta que procesa una cadena de símbolos y cambia de estado de acuerdo con una función de transición. Al finalizar el procesamiento, el autómata determina si la cadena es aceptada o rechazada.

Formalmente, un AFD se representa mediante la siguiente tupla:

$$AFD = (\Sigma, Q, \delta, q_0, F)$$

Donde:

- Σ : Es el alfabeto de entrada, formado por el conjunto de símbolos que puede procesar el autómata.
- Q : Es el conjunto finito y no vacío de estados del autómata.
- δ : Es la función de transición que determina el estado siguiente a partir del estado actual y el símbolo de entrada.

$$\delta : Q \times \Sigma \rightarrow Q$$

- q_0 : Es el estado inicial del autómata, perteneciente al conjunto Q .
- F : Es el conjunto de estados finales o de aceptación, donde $F \subseteq Q$.

Durante el procesamiento de una cadena, el autómata comienza en el estado inicial y consume los símbolos de entrada uno por uno. Para que una cadena sea aceptada, el autómata debe terminar en un estado perteneciente al conjunto de estados finales.

La implementación de un AFD en un lenguaje de programación permite representar sus componentes mediante estructuras de datos, variables y funciones. De esta manera, es posible simular el comportamiento del autómata y verificar diferentes cadenas de entrada.

Objetivos

Objetivo general

Construir un programa en un lenguaje de programación de alto nivel que implemente un Autómata Finito Determinista (AFD) para analizar un conjunto de cadenas de símbolos y determinar si pertenecen o no al lenguaje reconocido por el autómata.

Objetivos específicos

- Identificar los componentes que conforman un Autómata Finito Determinista.
- Implementar un AFD mediante un lenguaje de programación de alto nivel.
- Representar el alfabeto, los estados, las transiciones y los estados de aceptación del autómata.
- Procesar diferentes cadenas de símbolos para determinar si son aceptadas o rechazadas.
- Comprender el funcionamiento de la función de transición durante la simulación del AFD.

Desarrollo

Para realizar la práctica se desarrolló un programa en lenguaje Java capaz de implementar un Autómata Finito Determinista (AFD) y utilizarlo para determinar si diferentes cadenas pertenecen al lenguaje reconocido por el autómata.

En este caso, la estructura del AFD se encuentra definida directamente dentro del programa, por lo que el usuario no necesita introducir los estados ni las transiciones del autómata. El usuario únicamente proporciona la cantidad de cadenas que desea evaluar y posteriormente introduce cada una de ellas.

El autómata utilizado trabaja con el alfabeto

$$\Sigma = \{0, 1\}$$

y cuenta con tres estados:

$$Q = \{q_1, q_2, q_3\}$$

El estado inicial es q_1 y el estado de aceptación es q_2 . Las transiciones del autómata se almacenan en una matriz dentro del programa, donde las filas representan los estados y las columnas corresponden a los símbolos del alfabeto.

La tabla de transiciones utilizada es la siguiente:

	0	1
q_1	q_1	q_2
q_2	q_3	q_2
q_3	q_2	q_2

De acuerdo con las transiciones definidas, el lenguaje que se pretende reconocer se representa mediante la expresión:

$$L = \{0^*1(1|00|01)^*\}$$

Esta expresión permite identificar el tipo de cadenas que pueden ser aceptadas por el autómata. El símbolo 0^* representa una cantidad arbitraria de ceros al inicio, seguida de un símbolo 1. Posteriormente, pueden aparecer los bloques 1, 00 o 01 en cualquier combinación.

Funcionamiento del programa

Al iniciar la ejecución, el programa muestra información sobre el autómata que será utilizado. Se indica el alfabeto, el lenguaje reconocido, el estado inicial y el estado de aceptación. De esta manera, el usuario puede conocer las características del autómata antes de realizar las pruebas.

Posteriormente, el programa solicita la cantidad de cadenas que se desean verificar. Para cada cadena se realiza una simulación del AFD comenzando siempre desde el estado inicial q_1 .

Durante la simulación, el programa recorre la cadena símbolo por símbolo. Para cada carácter se determina la columna correspondiente dentro de la matriz de transiciones y se obtiene el siguiente estado del autómata.

Además, se verifica que todos los símbolos de la cadena pertenezcan al alfabeto $\Sigma = \{0, 1\}$. Si se encuentra un símbolo diferente de 0 o 1, la cadena se considera inválida y es rechazada.

Cuando se terminan de procesar todos los símbolos de la cadena, el programa compara el estado en el que terminó la simulación con el estado de aceptación q_2 . Si ambos coinciden, la cadena es aceptada; de lo contrario, es rechazada.

Los resultados de todas las cadenas se almacenan para mostrarlos posteriormente junto con la cadena que fue evaluada.

Procedimiento de evaluación

El funcionamiento general del programa puede resumirse en los siguientes pasos:

1. Definir dentro del programa los estados, el alfabeto, el estado inicial, el estado de aceptación y las transiciones del AFD.
2. Mostrar al usuario las características principales del autómata y el lenguaje que reconoce.
3. Solicitar la cantidad de cadenas que serán evaluadas.
4. Leer cada cadena introducida por el usuario.
5. Iniciar la simulación de cada cadena desde el estado inicial q_1 .
6. Recorrer los símbolos de la cadena de izquierda a derecha.
7. Consultar la matriz de transiciones para determinar el siguiente estado.
8. Verificar que los símbolos utilizados pertenezcan al alfabeto Σ .
9. Al finalizar la cadena, comprobar si el estado alcanzado corresponde al estado de aceptación q_2 .
10. Almacenar y mostrar el resultado correspondiente como **ACEPTADA** o **RECHAZADA**.

Estructura de datos utilizada

Para representar las transiciones del AFD se utiliza una matriz bidimensional de tipo entero:

```
int[] [] transiciones = {  
    {0, 0, 0},  
    {0, 1, 2},  
    {0, 3, 2},  
    {0, 2, 2}  
};
```

El índice de las filas corresponde al estado actual, mientras que las columnas representan los símbolos 0 y 1. El valor almacenado en cada posición indica el estado al que debe trasladarse el autómata.

También se utilizan arreglos para almacenar las cadenas introducidas y sus respectivos resultados. De esta forma, el programa puede mostrar al final cada cadena acompañada por el resultado de su evaluación.

Resultado de la ejecución

Una vez introducida la cantidad de cadenas, el programa solicita cada palabra al usuario. Al finalizar todas las evaluaciones, se muestra una sección de resultados en la que cada cadena aparece acompañada por el mensaje **ACEPTADA** o **RECHAZADA**.

Por ejemplo, para las cadenas:

101, 0, 11, 1000100001

el programa puede mostrar:

```
101 ACEPTADA
0 RECHAZADA
11 ACEPTADA
1000100001 ACEPTADA
```

De esta manera, el programa permite comprobar de forma automática si las cadenas introducidas por el usuario son reconocidas por el Autómata Finito Determinista definido en el código. ““

Resultados

Durante la realización de la práctica se implementó un programa destinado a reconocer cadenas mediante un Autómata Finito Determinista.

El programa permite procesar diferentes cadenas de entrada y determinar su pertenencia al lenguaje reconocido por el autómata. Para ello, se utilizan las transiciones definidas y se comprueba el estado final alcanzado durante la simulación.

““latex

Implementación del programa

A continuación se muestra el código fuente desarrollado en lenguaje Java para implementar el Autómata Finito Determinista descrito anteriormente. El programa contiene la definición de los estados y las transiciones del autómata, solicita al usuario las cadenas que desea evaluar y finalmente muestra el resultado de cada una.

Listing 1: Implementación del AFD en Java

```
import java.util.Scanner;
import java.io.*;
import java.util.*;

public class AFD {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int [][] transiciones = {
            {0, 0, 0},
            {0, 1, 2},
            {0, 3, 2},
            {0, 2, 2}
        };

        int estadoInicial = 1;
        int estadoAceptacion = 2;

        System.out.println("AUTOMATA-FINITO-DETERMINISTA");

        System.out.println();

        System.out.println(
            "El Automata-Finito-Determinista-esta-definido-"
            + "sobre-el-lenguaje"
        );
    }
}
```

```

System.out.println(
    "Sigma={0,1}"
);

System.out.println(
    "y acepta el conjunto de palabras del lenguaje:"
);

System.out.println(
    "L={0*1(1|00|01)*}"
);

System.out.println();

System.out.println(
    "Estado inicial: q1"
);

System.out.println(
    "Estado de aceptacion: q2"
);

System.out.println();

System.out.print(
    "Cuantas cadenas desea verificar?: "
);

int C = sc.nextInt();

String[] cadenas = new String[C];
boolean[] resultados = new boolean[C];

for (int i = 0; i < C; i++) {

    System.out.print(
        "Ingrese la cadena-" + (i + 1) + ": "
    );

    cadenas[i] = sc.next();

    int estadoActual = estadoInicial;
    boolean valida = true;

    for (int j = 0;
        j < cadenas[i].length();
        j++) {

        char simbolo = cadenas[i].charAt(j);

        if (simbolo != '0' && simbolo != '1') {
            valida = false;
            break;
        }
    }
}

```

```

        int columna;

        if (simbolo == '0') {
            columna = 1;
        } else {
            columna = 2;
        }

        estadoActual =
            transiciones[estadoActual][columna];
    }

    if (valida &&
        estadoActual == estadoAceptacion) {

        resultados[i] = true;

    } else {

        resultados[i] = false;
    }
}

System.out.println();

System.out.println("RESULTADOS");

for (int i = 0; i < C; i++) {

    if (resultados[i]) {

        System.out.println(
            cadenas[i] + " ACEPTADA"
        );

    } else {

        System.out.println(
            cadenas[i] + " RECHAZADA"
        );

    }
}

sc.close();
}
}

```

Cuestionario

```

run:

AUTOMATA FINITO DETERMINISTA

El Automata Finito Determinista esta definido sobre el lenguaje
Sigma = {0, 1}
y acepta el conjunto de palabras del lenguaje:
L = { 0*1(1|00|01)* }

Estado inicial: q1
Estado de aceptacion: q2

Cuantas cadenas desea verificar?: 2
Ingrese la cadena 1: 10001
Ingrese la cadena 2: 00000

RESULTADOS
10001 ACEPTADA
00000 RECHAZADA
BUILD SUCCESSFUL (total time: 12 seconds)
|

```

Figure 1: Enter Caption

¿Cuáles son los elementos que definen un AFD?

Un Autómata Finito Determinista se define mediante la tupla:

$$AFD = (\Sigma, Q, \delta, q_0, F)$$

Sus elementos son el alfabeto de entrada Σ , el conjunto finito de estados Q , la función de transición δ , el estado inicial q_0 y el conjunto de estados finales o de aceptación F .

Estos componentes determinan el funcionamiento del autómata y permiten establecer qué cadenas pertenecen al lenguaje reconocido.

¿Cuál es la utilidad de una tabla de transiciones de estado de un AFD?

La tabla de transiciones permite representar de manera organizada los movimientos del autómata. Indica a qué estado debe dirigirse el AFD cuando se encuentra en un estado determinado y recibe un símbolo específico del alfabeto.

Su utilidad principal es facilitar la definición de las transiciones y la implementación del autómata en un programa, ya que permite consultar de manera sistemática el siguiente estado durante el procesamiento de una cadena.

¿Qué importancia tienen los diagramas de transiciones de estado en el proceso de construcción de un AFD?

Los diagramas de transiciones de estado permiten visualizar gráficamente la estructura y el comportamiento de un Autómata Finito Determinista.

Cada nodo representa un estado y cada flecha representa una transición asociada a un símbolo de entrada. El estado inicial se identifica mediante una flecha de entrada y los estados finales se representan mediante círculos dobles.

Estos diagramas facilitan la comprensión de las transiciones, ayudan a detectar posibles errores en el diseño del autómata y permiten comprobar que los estados y sus conexiones corresponden con el lenguaje que se desea reconocer.

¿Cuáles son las ventajas de la simulación del AFD?

La simulación de un Autómata Finito Determinista permite comprobar su funcionamiento mediante diferentes cadenas de entrada. Entre sus ventajas se encuentran la verificación de las transiciones, la identificación de cadenas aceptadas y rechazadas y la detección de errores en la implementación.

Además, la simulación facilita la comprensión del recorrido que realiza el autómata entre sus estados y permite validar si el programa cumple con las condiciones establecidas para el lenguaje reconocido.

Conclusiones

La realización de esta práctica permitió comprender los componentes y el funcionamiento de un Autómata Finito Determinista, así como su implementación mediante un lenguaje de programación de alto nivel.

A través del desarrollo del programa se relacionaron los conceptos teóricos del AFD con su aplicación computacional, especialmente en la representación de estados, alfabetos, transiciones y estados de aceptación. El procesamiento de cadenas permitió identificar cómo la función de transición determina el recorrido del autómata y cómo el estado final establece si una cadena es aceptada o rechazada.

Finalmente, la práctica contribuyó al desarrollo de habilidades para implementar y analizar autómatas, reforzando los conocimientos relacionados con la teoría de lenguajes formales y la construcción de programas de reconocimiento de cadenas.

Bibliografía

References

- [1] Giró, J., Vázquez, J., Meloni, B. y Constable, L. (2015). *Lenguajes formales y teoría de autómatas*. Editorial Alfaomega. Argentina.
- [2] Ruiz Catalán, J. (2010). *Compiladores: teoría e implementación*. Editorial Alfaomega. México.
- [3] Brookshear, J. G. (1995). *Teoría de la Computación. Lenguajes formales, autómatas y complejidad*. Editorial Addison-Wesley Iberoamericana. Primera edición. U.S.A.